# COMP2271: Computer Graphics

Part 4: Geometry & Texture Mapping

Lecturer: Frederick Li
Department of Computer Science
Durham University

**How to Learn This Lecture**

This module transitions from "how it looks" to "how it is built." To master it, visualise the data flow:

- **Follow the Data:** Do not just read the code; trace the path of a vertex. It starts as a JavaScript array (RAM), moves to a Buffer Object (VRAM), and is interpreted by a Pointer before hitting the Shader.
- **The Blob vs. The Map:** Understand that a Vertex Buffer Object (VBO) is just a dumb "blob" of binary data. The `vertexAttribPointer` is the "map" that tells the GPU how to read that blob (e.g., "take 3 floats, skip 2 bytes").
- **Wrapping Paper:** Visualise Texture Mapping (UVs) exactly like wrapping a gift. The 3D model is the box, the Texture is the paper, and the UV coordinates are the instructions on where to fold and tape the paper to the box.

# 1 Geometry & Meshes: Constructing 3D Shapes

## 1.1 Introduction: The Blueprint of Virtual Worlds

Welcome to the foundational layer of 3D graphics. Before we can render a stunningly realistic scene, we must first describe the shapes of the objects within it, creating the very blueprint of our virtual world. Every complex 3D model, from a character in a video game to a detailed architectural rendering, is fundamentally composed of a collection of simple geometric primitives. In modern real-time graphics, this primitive is almost universally the triangle, chosen for its mathematical simplicity and guaranteed planarity.

This chapter delves into the "what" of our rendering pipeline. We will explore how 3D models are represented as data structures that a computer can understand and that a Graphics Processing Unit (GPU) can process efficiently. We will learn to define the surface of a model as a **Mesh** and understand the two critical components that form it: the **vertices** that define its points and the **indices** that describe how to connect them. By the end of this section, you will understand:

- The core concepts of a 3D mesh, vertices, and vertex attributes.
- The role and importance of index buffers for memory efficiency.
- How to create and manage Vertex Buffer Objects (VBOs) and Index Buffer Objects (IBOs) in WebGL to send geometry data to the GPU.
- The process of instructing the GPU on how to interpret this data using vertex attribute pointers.

- The final step of issuing a draw call to render the geometry.

Mastering this data flow is crucial, as it forms the bedrock upon which all other visual effects are built. Transformations, lighting, and texturing are all operations performed on the geometric data we define here.

## 1.2 The Anatomy of a 3D Model: The Mesh

**Definition: Mesh**

A **Mesh** is the collection of vertices, edges, and faces that define the shape of a polyhedral object in 3D computer graphics and solid modelling. In essence, it is the digital scaffolding that creates the surface of a 3D model.

At its heart, a mesh is defined by two primary lists of data:

1. **Vertices**: This is a comprehensive list of all the unique points that constitute the shape. However, a vertex is much more than just a 3D position $(x, y, z)$. It is a collection of **attributes**. For a vertex to be useful in a modern rendering pipeline, it typically includes:
   - **Position**: The mandatory $(x, y, z)$ coordinates in 3D space.
   - **Normal Vector**: A vector pointing perpendicularly outwards from the surface at that vertex. This is crucial for lighting calculations.
   - **Texture Coordinates (UVs)**: A 2D coordinate $(u, v)$ that maps this vertex to a specific point on a 2D texture image.
   - **Colour**: A per-vertex colour attribute, which can be used for simple colouring effects or interpolation.

   This data is stored on the GPU in a memory buffer known as a **Vertex Buffer Object (VBO)**.

2. **Indices**: This is a separate list, composed entirely of integers. These integers correspond to the indices (or positions) of the vertices in the vertex list. They provide the instructions for connecting the vertices to form triangles. For example, the list ‘[0, 1, 2]‘ instructs the GPU to form a triangle using the 1st, 2nd, and 3rd vertices from the vertex buffer (remembering that lists are 0-indexed). This list is stored on the GPU in an **Index Buffer Object (IBO)**.

### 1.2.1 Why Use an Index Buffer? Efficiency is Key

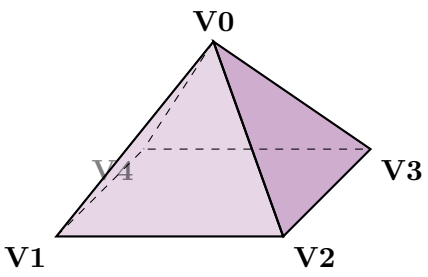
Using an index buffer is a critical optimisation. Consider a simple square composed of two triangles. Without an index buffer, you would need to define six vertices in total, as the two vertices along the shared diagonal would be duplicated. With an index buffer, you only need to define the four unique corner vertices. The index buffer then references these four vertices to create the two triangles.

- **Without Indices**: 6 vertices × (Position + Normal + UV) = Large memory footprint.
- **With Indices**: 4 vertices + 6 indices = Significantly smaller memory footprint.

This memory saving becomes enormous for complex models with thousands or millions of triangles, reducing the amount of data that needs to be sent to and stored on the GPU.

### 1.2.2 Visualising the Mesh Structure

The diagram below illustrates how vertices and indices work together to form a simple 3D shape (a pyramid). Notice how vertex 'V0' (the apex) is shared by three separate visible triangles, making it a perfect candidate for reuse via an index buffer.




**Vertex Buffer (VBO)**

```
0:  V0 (x,y,z...)
1:  V1 (x,y,z...)
2:  V2 (x,y,z...)
3:  V3 (x,y,z...)
4:  V4 (x,y,z...)
```

**Index Buffer (IBO)**

```
% Front Face
[0, 1, 2]
% Right Face
[0, 2, 3]
% ...and so on
```

Figure 1: A 3D pyramid mesh constructed from a vertex buffer and an index buffer. The index buffer defines triangles by referencing vertices, allowing for efficient reuse.

## 1.3 From CPU to GPU: Buffer Objects in WebGL

The vertex and index data we define in JavaScript exists in the computer's main memory (RAM), which is managed by the CPU. Since the GPU operates on its own high-speed memory (VRAM), our first task is to transfer this geometric data across that hardware divide. WebGL provides buffer objects for this purpose.

### 1.3.1 Defining Vertex Data in JavaScript

First, we define our geometry in a standard JavaScript array. For performance, we often use an **interleaved** format, where all attributes for a single vertex are placed contiguously in the array. This improves memory access patterns (cache locality) on the GPU.

```
// A simple square with position (X, Y) and color (R, G, B) attributes.
// Data is interleaved: [X1, Y1, R1, G1, B1, X2, Y2, R2, G2, B2, ...]
const vertices = [
  // X,    Y,    R,   G,   B
   0.5,  0.5,   1.0, 0.0, 0.0, // Top-right (red)
   0.5, -0.5,   0.0, 1.0, 0.0, // Bottom-right (green)
  -0.5, -0.5,   0.0, 0.0, 1.0, // Bottom-left (blue)
  -0.5,  0.5,   1.0, 1.0, 0.0, // Top-left (yellow)
];
```

### 1.3.2 Creating a Vertex Buffer Object (VBO)

Next, we create a VBO and copy our JavaScript array data into it. This is a three-step process:

1. **Create Buffer**: `gl.createBuffer()` requests a new buffer object handle from WebGL.
2. **Bind Buffer**: `gl.bindBuffer(gl.ARRAY_BUFFER, ...)` activates the buffer, making it the current target for subsequent buffer operations on the `ARRAY_BUFFER` binding point.
3. **Buffer Data**: `gl.bufferData(...)` allocates and copies the data. We must convert our JavaScript array into a `Float32Array`, which is a typed array with a memory layout that the GPU can understand directly. The `gl.STATIC_DRAW` hint tells WebGL that we intend to load the data once and use it many times, allowing for potential optimisations.

```
// 1. Create a buffer object on the GPU
const vertexBuffer = gl.createBuffer();
// 2. Bind the buffer as the current ARRAY_BUFFER
gl.bindBuffer(gl.ARRAY_BUFFER, vertexBuffer);
// 3. Copy the JavaScript vertex data into the bound buffer
gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(vertices), gl.STATIC_DRAW);
```

### 1.3.3 Creating an Index Buffer Object (IBO)

The process for an IBO is almost identical, but we use a different binding point (`gl.ELEMENT_ARRAY_BUFFER`) and a different typed array (`Uint16Array` for integers).

```
// We use the same 'vertices' array with 4 unique vertices as defined before.
// Now, define indices to form two triangles (0->1->2 and 0->2->3)
const indices = [0, 1, 2,  0, 2, 3];
// 1. Create the index buffer
const indexBuffer = gl.createBuffer();

// 2. Bind the buffer as the current ELEMENT_ARRAY_BUFFER
gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, indexBuffer);
// 3. Copy the JavaScript index data into the bound buffer
gl.bufferData(gl.ELEMENT_ARRAY_BUFFER, new Uint16Array(indices), gl.STATIC_DRAW);
```

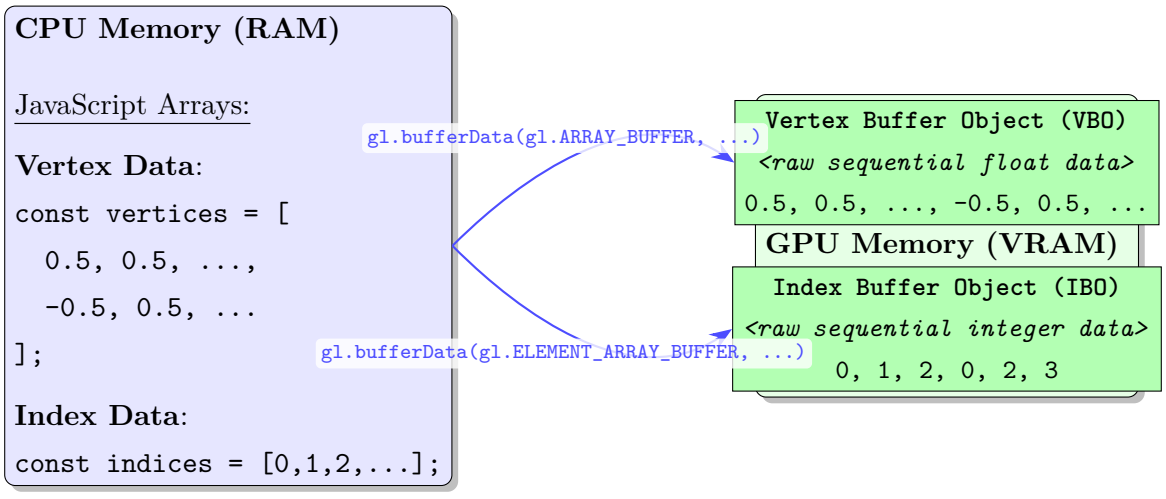



Figure 2: The process of transferring vertex and index data from CPU memory to dedicated buffer objects (VBO and IBO) in GPU memory.

## 1.4 Describing the Data: Vertex Attribute Pointers

Once the data is on the GPU, we must explicitly tell the vertex shader how to interpret the raw, interleaved data within the VBO. This is done by setting up **vertex attribute pointers**. Each 'attribute' in our vertex shader (e.g., `a_position`, `a_color`) needs a pointer that specifies its layout within the buffer. The `gl.vertexAttribPointer()` function is the key here. It answers the following questions for a single attribute:

- **Which attribute?**: The location of the shader attribute.
- **How many components?**: The size of the attribute (e.g., 3 for a `vec3` position, 2 for a `vec2` UV coordinate).
- **What is the data type?**: e.g., `gl.FLOAT`.
- **Should it be normalised?**: `false` for floats.
- **What is the stride?**: How many bytes to advance from the start of one vertex to the start of the next. For interleaved data, this is the total size of all attributes for one vertex.
- **What is the offset?**: How many bytes from the start of a vertex's data does this specific attribute begin?.

### 1.4.1 Example: Setting Pointers for Interleaved Data

Let's revisit our square with position (2 floats) and colour (3 floats).

```
// Interleaved data: [X1, Y1, R1, G1, B1, X2, Y2, R2, G2, B2, ...]
// Get attribute locations from the compiled shader program
const positionAttribLocation = gl.getAttribLocation(program, 'a_position');
const colorAttribLocation = gl.getAttribLocation(program, 'a_color');

// Bind the VBO before setting pointers, to associate the pointers with this buffer
gl.bindBuffer(gl.ARRAY_BUFFER, vertexBuffer);
// -- Define the layout --
const FSIZE = Float32Array.BYTES_PER_ELEMENT;
// Stride: 5 floats (2 for pos, 3 for color) per vertex
const stride = 5 * FSIZE;
// --- Set up the Position Attribute Pointer ---
// It has 2 components, starts at an offset of 0
gl.vertexAttribPointer(
    positionAttribLocation, // Attribute location
    2,                      // Number of elements per attribute (vec2)
```

```
    gl.FLOAT,               // Type of elements
    false,                  // Normalised?
    stride,                 // Stride - bytes from one vertex to the next
    0                       // Offset - bytes from the start of a vertex
);
// --- Set up the Color Attribute Pointer ---
// It has 3 components, starts after the position data (offset of 2 floats)
gl.vertexAttribPointer(
    colorAttribLocation,
    3,
    gl.FLOAT,
    false,
    stride,
    2 * FSIZE // Offset
);
// -- Finally, enable the attributes for use during rendering --
gl.enableVertexAttribArray(positionAttribLocation);
gl.enableVertexAttribArray(colorAttribLocation);
```

**Interleaved Vertex Buffer in GPU Memory:**

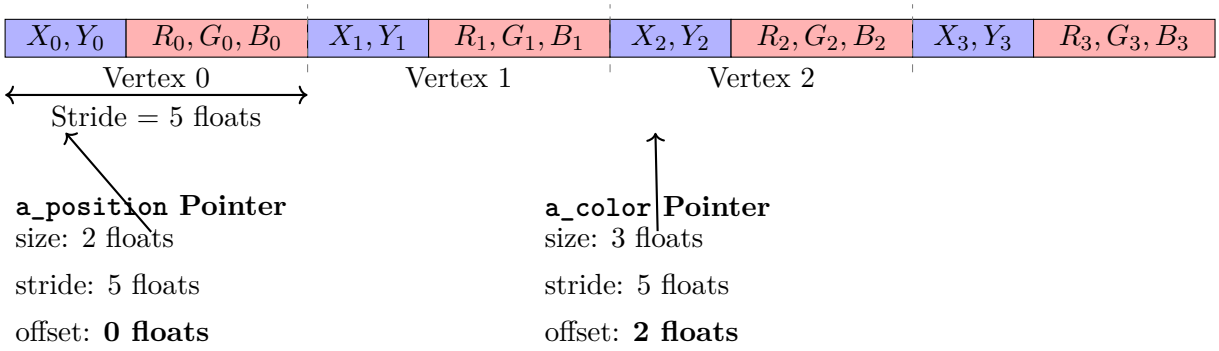



Figure 3: Visual representation of how `vertexAttribPointer` interprets an interleaved VBO. The stride defines the size of a whole vertex block, while the offset specifies where each attribute starts within that block.

## 1.5 The Grand Finale: The Draw Call

After all the setup is completeshaders compiled, buffers created and filled, and attribute pointers configureda single command is all that's needed to trigger the entire rendering pipeline. The specific draw call you use depends on whether or not you are using an index buffer.

### 1.5.1 Drawing Without an Index Buffer: `gl.drawArrays`

This command reads vertices sequentially from the currently bound `ARRAY_BUFFER`.

**`gl.drawArrays(mode, first, count)`**

- `mode`: The type of primitive to render, e.g., `gl.TRIANGLES`.
- `first`: The starting index in the enabled arrays.
- `count`: The total number of vertices to be rendered.

```
// To draw a square (2 triangles) without an index buffer, we need 6 vertices.
gl.drawArrays(gl.TRIANGLES, 0, 6);
```

### 1.5.2 Drawing With an Index Buffer: `gl.drawElements`

This command is more efficient for complex meshes. It uses the currently bound `ELEMENT_ARRAY_BUFFER` to determine which vertices to fetch from the `ARRAY_BUFFER`.

**`gl.drawElements(mode, count, type, offset)`**

- `mode`: The type of primitive, e.g., ‘gl.TRIANGLES‘.
- `count`: The total number of indices to be rendered from the element array buffer.
- `type`: The data type of the values in the element array buffer (e.g., `gl.UNSIGNED_SHORT`).
- `offset`: An offset in bytes into the element array buffer.

```
// To draw the same square, we use 6 indices that reference our 4 vertices.
// The index buffer must be bound before this call.
gl.drawElements(gl.TRIANGLES, 6, gl.UNSIGNED_SHORT, 0);
```

This single command initiates the pipeline: the GPU fetches the specified vertices (using the indices if provided), runs the vertex shader for each one, rasterizes the triangles, and finally runs the fragment shader for every pixel within them.

### 1.5.3 The High-Level Abstraction: Creating a Mesh in Three.js

While understanding the low-level WebGL process is crucial, modern libraries like **Three.js** abstract away the manual buffer management, attribute pointers, and draw calls. This allows developers to focus on scene composition rather than GPU communication. Below is the equivalent of creating and displaying our indexed square.

```
import * as THREE from 'three';
// 1. Define the Geometry
// Three.js provides convenience classes for common shapes.
// This single line creates the vertices, indices, normals, and UVs for a plane.
const geometry = new THREE.PlaneGeometry(1, 1);
// 2. Define the Material (the "paint")
// This material will apply a uniform color and is not affected by lights.
const material = new THREE.MeshBasicMaterial({ color: 0x00ff00 }); // Green

// 3. Create the Mesh
// The Mesh object combines the shape (geometry) and its appearance (material).
const squareMesh = new THREE.Mesh(geometry, material);

// 4. Add to Scene
// 'scene' is a top-level Three.js object.
// Adding the mesh to the scene ensures it will be rendered on the next render call.
scene.add(squareMesh);
// In your render loop:
// renderer.render(scene, camera);
// This single call handles all the binding, attribute setup, and draw calls
↪  internally.
```

In this example, Three.js automatically performs the steps we did manually in WebGL: it creates VBOs and IBOs from the ‘PlaneGeometry‘ data, compiles appropriate shaders based on the ‘MeshBasicMaterial‘, binds the buffers, sets up attribute pointers, and issues a ‘gl.drawElements‘ call inside the ‘renderer.render‘ function.

# 2 Texture Mapping: Adding Detail to Surfaces

## 2.1 Introduction: Painting on 3D Models

So far, the objects we can render are defined by their shape and can be assigned simple, uniform colours. To create visually rich and believable worlds, we need to add surface detailthe grain of wood, the roughness of brick, the intricate design on a character's clothing. Adding this level of detail by creating more triangles would be computationally prohibitive and artistically tedious.

This is where **Texture Mapping** comes in. It is a revolutionary technique that allows us to "wrap" a 2D image, called a **texture**, onto the surface of a 3D model. This method decouples the surface appearance from the geometric complexity, enabling us to render highly detailed objects with relatively simple underlying geometry. In this chapter, we will explore:

- The core concept of a texture and the UV mapping system used to apply it.
- How to load image data in JavaScript and prepare it as a WebGL texture.
- The process of **sampling** a texture within the fragment shader.
- Critical texture parameters like filtering and wrapping, which control the quality and appearance of the texture on the model.
- How to integrate texture colour with the lighting models learned previously to create realistically shaded and detailed surfaces.

## 2.2 Fundamental Mapping Strategies

Before understanding how we define coordinates, we must understand the fundamental algorithm used to connect a 2D image to a 3D surface. Mathematically, there are two ways to approach this problem: **Forward Mapping** and **Backward Mapping**.

### 2.2.1 Forward Mapping (Source Texture to Projected Geometry on Screen)

In this approach, the algorithm iterates over every pixel in the **source texture image** (texel). For each texel, it calculates where that point would land on the final screen geometry and "splats" the colour there.

$$(u, v) \xrightarrow{\text{Project}} (x, y)$$

**The Problem (Holes):** This method creates significant visual artifacts. If the texture is small (e.g., $64 \times 64$) but the object appears large on the 4K screen, the source texels will spread apart, leaving visible gaps or "holes" between the coloured pixels on the screen. The texture does not cover the surface continuously.

### 2.2.2 Backward Mapping (Projected Geometry on Screen to Source Texture)

This is the inverse approach, and it is the standard used by all modern rasterisation hardware (including WebGL and Three.js). The algorithm iterates over every **pixel on the destination geometry on screen**. For each pixel, it calculates the "Inverse Function" to find which specific point on the source texture corresponds to the pixel's center.

$$(x, y) \xrightarrow{\text{Inverse}} (u, v)$$

**The Benefit (Guaranteed Coverage):** Because we are driven by the screen pixels, we are mathematically guaranteed that every single pixel will find a color value. There are no holes, regardless of how close or far the camera is to the object.

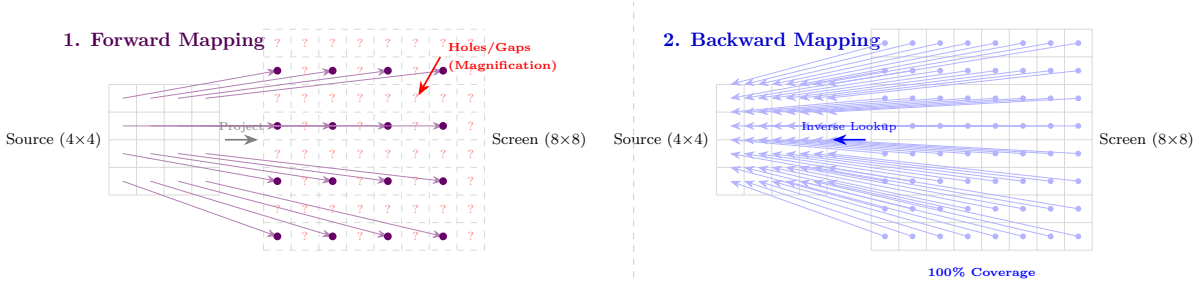



Figure 4: Detailed Comparison of Mapping Strategies. **Left:** Forward mapping scatters source pixels onto the screen, creating holes (red '?') when zooming in. **Right:** Backward mapping iterates over every screen pixel and reaches back to fetch the correct color, ensuring no gaps appear.

## 2.3 The Core Concept: UV Mapping

**Definition: Texture**

A **Texture** is a 2D image that is applied to the surface of a 3D model to add colour, detail, or other material properties. The individual pixels of a texture are often called **texels** to distinguish them from screen pixels.

To control how this 2D image wraps around a 3D model, we need a mapping system. This is

achieved by assigning a 2D coordinate to each 3D vertex of the mesh. These 2D coordinates are called **UV coordinates**.

- The **U** coordinate corresponds to the horizontal axis (X-axis) of the 2D texture image.
- The **V** coordinate corresponds to the vertical axis (Y-axis) of the 2D texture image.

Both U and V coordinates are typically normalised to a range of $[0, 1]$, where $(0, 0)$ is often the bottom-left corner of the texture and $(1, 1)$ is the top-right. The process of creating this mappingessentially "unwrapping" the 3D model into a flat 2D patternis called **UV unwrapping** and is a crucial step performed by 3D artists in modelling software.

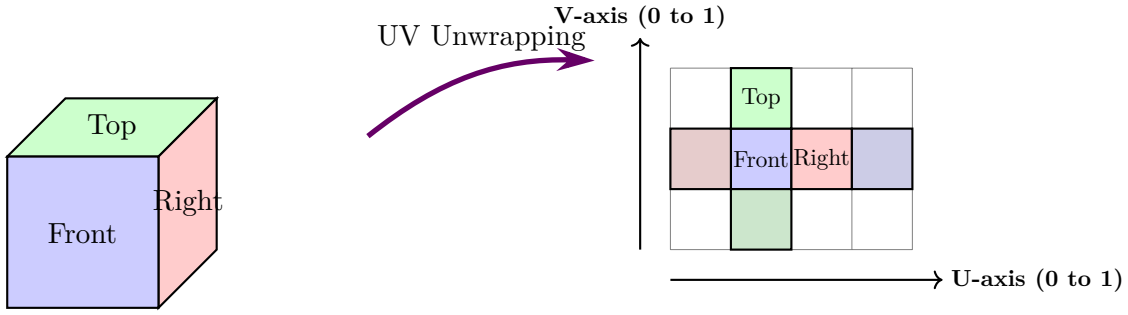



Figure 5: A 3D cube is "unwrapped" into a 2D UV layout. Each vertex on the 3D model has a corresponding (u, v) coordinate on this 2D map, which tells the GPU where to look on the texture image.

These UV coordinates are stored as another vertex attribute and are passed from the vertex shader to the fragment shader. During rasterisation, the GPU smoothly interpolates these coordinates across the surface of the triangle, providing a unique UV coordinate for every single fragment (pixel).

## 2.4 Working with Textures in WebGL

### 2.4.1 Passing UV Coordinates

The first step is to treat the UV coordinates like any other vertex attribute. They are defined in our vertex buffer and passed as a ‘varying‘ from the vertex shader to the fragment shader.

```
// --- Vertex Shader ---
attribute vec2 a_texCoord; // UV attribute from the VBO
varying vec2 v_texCoord;
// Pass-through to the fragment shader
```

```
5
6  void main() {
7    // ... transform gl_Position ...
8
9    // Simply pass the texture coordinate to the fragment shader.
10   // The rasterizer will interpolate it across the triangle's surface.
11   v_texCoord = a_texCoord;
12 }
```

### 2.4.2 Loading the Texture in JavaScript

Loading an image from a server is an **asynchronous** operation. This means we cannot halt execution and must wait for the image to finish downloading before we can upload it to the GPU. This is typically handled using the 'image.onload' event handler.

The process involves:

1. Creating a WebGL texture object with `gl.createTexture()`.
2. Creating an HTML 'Image' object and setting its `src` to begin the download.
3. In the `image.onload` callback function, we bind the texture and upload the now-available image data to it using `gl.texImage2D()`.
4. Optionally, but highly recommended, generate mipmaps with `gl.generateMipmap()` for better rendering quality when the texture is viewed from a distance.

```
1  // Create a WebGL texture object
2  const texture = gl.createTexture();
3  gl.bindTexture(gl.TEXTURE_2D, texture);
4  // Create a JavaScript Image object
5  const image = new Image();
6  // IMPORTANT: All GPU-related texture operations must happen inside this callback.
7  image.onload = function() {
8    // Bind the texture again as the active one
9    gl.bindTexture(gl.TEXTURE_2D, texture);
10   // Upload the loaded image data to the GPU texture object
11   gl.texImage2D(
12       gl.TEXTURE_2D, 0, gl.RGBA, gl.RGBA,
```

```
      gl.UNSIGNED_BYTE, image
  );
  // Generate mipmaps for better minification quality
  gl.generateMipmap(gl.TEXTURE_2D);

  // You might need to re-render the scene here if the image loads after initial
  ↪ render
};
// Start the asynchronous image download
image.src = 'path/to/my_texture.png';
```

### 2.4.3 Sampling the Texture in the Fragment Shader

The final step happens in the fragment shader. Here, we use the interpolated `v_texCoord` to look up the colour from the texture. This process is called **sampling**. The fragment shader receives the texture through a special uniform type called `sampler2D`. The built-in GLSL function `texture2D()` performs the lookup.

```
// --- Fragment Shader ---
precision mediump float;
// Varying received from the vertex shader (interpolated)
varying vec2 v_texCoord;

// Uniform representing the texture image from JavaScript
uniform sampler2D u_texture;
void main() {
  // Sample the texture at the given UV coordinate
  vec4 textureColor = texture2D(u_texture, v_texCoord);
  // Set the fragment's final color to the color from the texture
  gl_FragColor = textureColor;
}
```

## 2.5 Controlling Texture Appearance: Filtering and Wrapping

The GPU rarely samples a texture at a perfect 1:1 pixel-to-texel ratio. When a 3D surface is close to the camera (**magnification**) or far away (**minification**), the GPU needs rules on how

to determine the sampled colour.

### 2.5.1 Texture Filtering

Filtering determines the colour when a UV coordinate falls between texels.

- `gl.NEAREST`: The simplest and fastest method. It grabs the colour of the single nearest texel. This is efficient but results in a blocky, pixelated look during magnification.
- `gl.LINEAR`: A smoother method, also known as bilinear filtering. It takes the four nearest texels and returns a weighted average of their colours based on the sample's proximity to each. This looks much better when magnified but can appear slightly blurry.

For minification (when the object is far away), using mipmaps with linear filtering (`gl.LINEAR_MIPMAP_LINEAR`, or trilinear filtering) provides the highest quality by sampling from two mipmap levels and blending the results.

```
// Set filtering parameters on the texture object
// Magnification filter (when texture is stretched)
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MAG_FILTER, gl.LINEAR);
// Minification filter (when texture is shrunk)
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER, gl.LINEAR_MIPMAP_LINEAR);
```

### 2.5.2 Texture Wrapping

Wrapping determines the behaviour when a UV coordinate falls outside the standard $[0, 1]$ range.

- `gl.REPEAT`: The texture repeats itself, tiling across the surface. This is perfect for patterns like bricks or wood grain.
- `gl.CLAMP_TO_EDGE`: The coordinate is clamped to the edge, causing the border colour of the texture to be stretched outwards.
- `gl.MIRRORED_REPEAT`: The texture repeats, but every other repetition is mirrored, which can help hide seams.

```
// Set wrapping parameters for both U (S) and V (T) axes
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_S, gl.REPEAT);
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_T, gl.REPEAT);
```

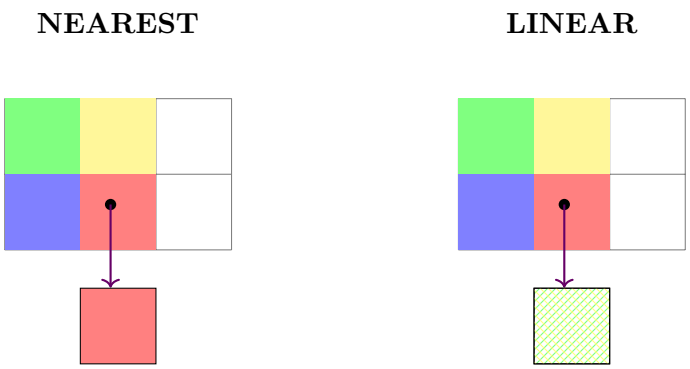



Figure 6: Comparison of 'NEAREST' filtering (left), which picks the closest texel, and 'LINEAR' filtering (right), which interpolates between the four neighbours, resulting in a smoother blend.

## 2.6 Combining Textures and Lighting

Textures are most powerful when they are integrated into our lighting model. A texture shouldn't make an object look flat; it should define the base colour of the surface that then reacts to light realistically. The standard approach is to sample the texture to get the object's base colour (also known as its **albedo**) and then modulate (multiply) this colour by the calculated light intensity from our lighting model (e.g., Blinn-Phong). The fragment shader now combines concepts from multiple lectures:

1. It receives interpolated normals and positions for **shading**.
2. It receives interpolated UV coordinates for **texturing**.
3. It calculates the total lighting intensity (ambient + diffuse + specular).
4. It samples the texture to get the surface colour.
5. It multiplies the lighting by the texture colour to get the final result.

```
// --- Fragment Shader with Texturing and Lighting ---
precision mediump float;

// Varyings
```

```
varying vec2 v_texCoord;
varying vec3 v_normal;
varying vec3 v_fragPosition;

// Uniforms
uniform sampler2D u_texture;
uniform vec3 u_lightDirection;
uniform vec3 u_lightColor;
// ... other uniforms for specular, view position, etc.

void main() {
  // 1. Sample the texture to get the material's base color (albedo)
  vec4 albedo = texture2D(u_texture, v_texCoord);
  // 2. Perform lighting calculations as before
  // Ambient
  vec3 ambient = 0.1 * u_lightColor;
  // Diffuse
  vec3 norm = normalize(v_normal);
  float diff = max(dot(norm, u_lightDirection), 0.0);
  vec3 diffuse = diff * u_lightColor;
  // Specular (simplified)
  // ... calculate specular component 'specular' ...

  // 3. Combine lighting components
  vec3 lighting = ambient + diffuse;
  // + specular;

  // 4. Modulate the lighting by the texture color
  vec3 finalColor = lighting * albedo.rgb;
  gl_FragColor = vec4(finalColor, albedo.a);
}
```

### 2.6.1 Texture Mapping in Three.js: A Simpler Workflow

Just as with geometry, Three.js greatly simplifies the process of loading and applying textures. It handles the asynchronous loading, ‘onload‘ callbacks, and shader uniform management internally.

```
import * as THREE from 'three';
// Assume 'scene' and a 'cubeMesh' (created from BoxGeometry) already exist.
```

```
// 1. Create a Texture Loader
// This object handles the asynchronous loading of images.
const textureLoader = new THREE.TextureLoader();
// 2. Load the texture
// The .load() method starts the download and returns a Texture object immediately.
// Three.js handles the update automatically once the image is ready.
const colorTexture = textureLoader.load('path/to/my_texture.png');
// 3. (Optional) Set Filtering and Wrapping
// These properties on the Texture object correspond directly to WebGL parameters.
colorTexture.magFilter = THREE.LinearFilter;
colorTexture.minFilter = THREE.LinearMipmapLinearFilter;
colorTexture.wrapS = THREE.RepeatWrapping;
colorTexture.wrapT = THREE.RepeatWrapping;
// 4. Create a Material with the Texture
// We use a physically-based material that reacts to light.
// The 'map' property tells the material to use our texture as the base color
↪  (albedo).
const material = new THREE.MeshStandardMaterial({
    map: colorTexture
});
// 5. Apply the material to the mesh
const texturedMesh = new THREE.Mesh(
    new THREE.BoxGeometry(1, 1, 1),
    material
);
scene.add(texturedMesh);

// When renderer.render() is called, Three.js will bind the texture,
// pass it to the 'sampler2D' uniform in the shader, and calculate the
// final color using the texture and any lights in the scene.
```

This high-level approach encapsulates all the WebGL stepscreating texture objects, handling image loading, setting parameters, and managing shader uniformsinto a clear and declarative workflow.

## 2.7 Mip-Mapping: Optimising for Distance

### 2.7.1 The Problem: Minification and Aliasing

When a texture is viewed close up, one screen pixel might cover a tiny fraction of a texel (magnification). However, when a textured object is far away, a single screen pixel might cover

**hundreds** of texels (minification).

If the GPU simply picks one texel (Nearest) or averages four (Linear), it ignores the vast majority of the texture data covered by that pixel. This leads to:

- **Visual Noise (Aliasing):** High-frequency details (like a checkerboard) shimmer and flicker as the camera moves.
- **Performance Waste:** The GPU cache thrashes because it tries to fetch texels that are widely spaced in memory.

### 2.7.2 The Solution: The Mip-Map Pyramid

**Mip-mapping** (from the Latin *multum in parvo*, "much in little") solves this by pre-calculating a sequence of lower-resolution versions of the texture.

- **Level 0:** Original Image ($W \times H$)
- **Level 1:** Half resolution ($W/2 \times H/2$)
- **Level 2:** Quarter resolution ($W/4 \times H/4$)
- ...down to $1 \times 1$.

During rendering, the GPU calculates the "Level of Detail" (LOD) based on the distance to the object and samples from the most appropriate mip-level.

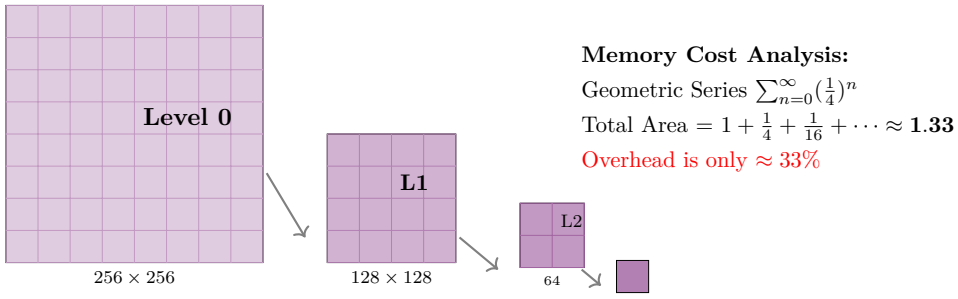



Figure 7: The Mip-Map Chain. Each level is half the width and height of the previous one. The total memory overhead for storing all smaller levels is roughly 33% of the original image.

### 2.7.3 Implementation details

In WebGL, generating mipmaps is a single command. However, there is a restriction: generally, the texture dimensions must be a **power of two** ($2^n$, e.g., 64, 128, 256, 512) for this to work automatically.

```
// 1. Upload the base image (Level 0)
gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA, gl.RGBA, gl.UNSIGNED_BYTE, image);

// 2. Automatically generate all smaller levels (Level 1 to N)
// GPU hardware performs the downscaling/averaging instantly.
gl.generateMipmap(gl.TEXTURE_2D);

// 3. Set the Minification Filter to use the Mipmaps
// gl.LINEAR_MIPMAP_LINEAR (Trilinear filtering) mixes values from
// two adjacent mip-levels for the smoothest possible result.
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER, gl.LINEAR_MIPMAP_LINEAR);
```

## 2.8 Procedural Texturing: HTML5 Canvas2D

While loading static images (PNG/JPG) is standard, sometimes we need dynamic textureslike a scoreboard that updates every second, or a text label that acts as a billboard. We can use the HTML5 2D Canvas API to "draw" a texture using the CPU, and then upload that canvas to the GPU.

### 2.8.1 How it Works

The ‘<canvas>‘ element is essentially a grid of pixels in RAM. The standard 2D Context allows us to draw shapes, text, and gradients easily. WebGL can accept an HTMLCanvasElement directly in ‘texImage2D‘, just like an Image object.

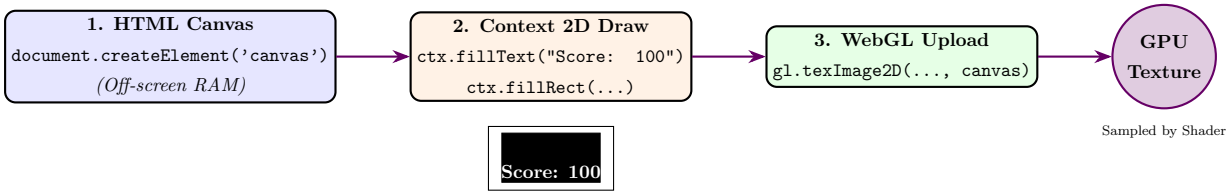



Figure 8: The pipeline for generating dynamic textures. The CPU constructs the image on a hidden canvas, which is then uploaded to VRAM as a standard texture.

### 2.8.2 Code Example: Creating a Text Billboard

This technique is widely used to put text labels above 3D characters.

```javascript
function createTextTexture(text) {
    // 1. Create an off-screen canvas
    const canvas = document.createElement('canvas');
    canvas.width = 256;  // Power of 2 is best!
    canvas.height = 128;
    const ctx = canvas.getContext('2d');

    // 2. Draw using standard HTML5 2D API
    ctx.fillStyle = '#000000';   // Black background
    ctx.fillRect(0, 0, 256, 128);

    ctx.fillStyle = '#FFFFFF';   // White text
    ctx.font = '48px Arial';
    ctx.textAlign = 'center';
    ctx.textBaseline = 'middle';
    ctx.fillText(text, 128, 64); // Draw text in center

    // 3. Create WebGL Texture
    const texture = gl.createTexture();
    gl.bindTexture(gl.TEXTURE_2D, texture);

    // 4. Upload the Canvas directly
    // Note: We use the canvas object itself as the data source
    gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA, gl.RGBA, gl.UNSIGNED_BYTE, canvas);

    // 5. Setup parameters (Standard filtering)
    gl.generateMipmap(gl.TEXTURE_2D);

    return texture;
}

// Usage:
// const scoreTexture = createTextTexture("Score: 9000");
```

**Performance Warning**

Drawing text to a canvas is fast, but **uploading** it to the GPU (`texImage2D`) is relatively slow because it involves moving data across the CPU-GPU bus. Do not update a canvas texture every single frame (60fps) unless absolutely necessary. Instead, update it only when the text changes (e.g., when the score actually increases).

# 3 The Aliasing Problem & Solutions

One of the most persistent artefacts in computer graphics is the "staircase effect" or "jaggies" seen along the edges of geometry. In technical terms, this is known as **Aliasing**. This phenomenon occurs because we are attempting to represent a continuous mathematical world using a discrete, finite grid of pixels.

## 3.1 The Problem: Sampling and Discrete Pixels

A computer screen is composed of a discrete grid of pixels, whereas the mathematical lines and triangles we define in our 3D scenes are continuous and have infinite precision. To render an image, the GPU must "sample" this continuous geometry at regular intervals to decide which pixels to colour.

- **The Cause:** The standard rasterisation process typically tests the **centre** of each pixel. If this single centre point lies inside a triangle, the entire pixel is coloured. If it lies even a fraction of a millimetre outside, the pixel remains empty.
- **The Result:** A diagonal line passing through this pixel grid will inevitably be represented as a jagged set of blocks rather than a smooth edge. This is fundamentally a signal processing failure: our "sampling rate" (the screen resolution) is too low to adequately capture the "frequency" (the sharp edges) of the signal, resulting in a loss of information and visual noise.

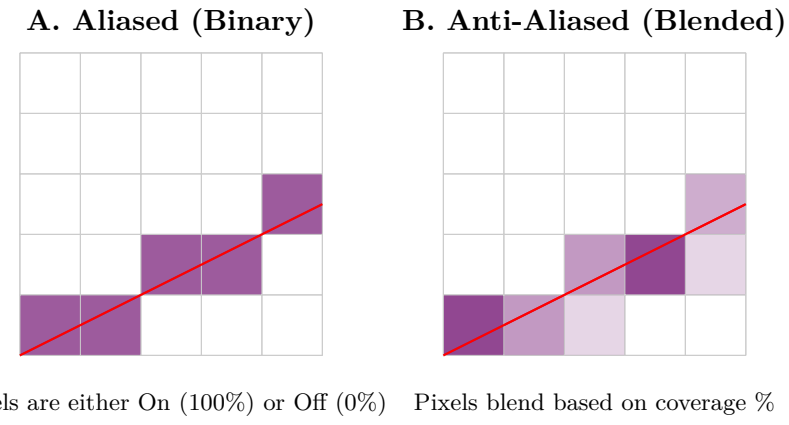



Figure 9: Comparison of an Aliased line (jagged) versus an Anti-Aliased line (smooth).

## 3.2 Solution 1: Supersampling (SSAA)

This is the "Brute Force" approach to solving the problem.

- **Method:** The engine renders the entire scene at a much higher resolution (e.g., 4× the actual screen size) in an off-screen buffer. It then shrinks this image down ("downsamples") to the final screen resolution.
- **Result:** A pixel that was only 50% covered by a triangle edge will naturally average out to 50% brightness when the 4 high-res pixels are combined into one.
- **Cost:** This method is extremely expensive. Rendering 4× the number of pixels means 4× the Fragment Shader load and 4× the memory bandwidth. Consequently, it is rarely used in real-time games today, except for high-quality screenshots or in older titles on modern hardware.

### 3.3 Solution 2: Multisample Anti-Aliasing (MSAA)

This technique represents the standard hardware optimisation of Supersampling, designed to balance quality with performance.

- **The Optimisation:** Instead of running the expensive Fragment Shader (lighting, textures, etc.) 4 times per pixel, MSAA runs it **only once** at the pixel centre.
- **Sub-Sampling:** However, the hardware stores Depth and Stencil values at 4 (or more) discrete "sub-sample" locations within that single pixel.
- **Coverage Logic:** If a triangle edge covers only 2 of the 4 sub-samples, the hardware understands that the pixel is "50% covered." It then mixes the single calculated colour with the background colour based on this coverage percentage.
- **Benefit:** This produces smooth geometric edges without the massive computational cost of shading the inside of the triangle 4 times.
- **Limitation:** Importantly, MSAA only smoothes geometric edges. It does not smooth aliasing found within textures or shader effects (such as sharp specular highlights), as the shader is still only run once per pixel.

**Implementation in Three.js:** In WebGL applications, MSAA is handled automatically by the browser's underlying context if requested during setup.

```
// Enable MSAA (usually on by default in Three.js)
const renderer = new THREE.WebGLRenderer({
    antialias: true
});
```

### 3.4 Solution 3: Fast Approximate Anti-Aliasing (FXAA)

As rendering techniques became more complex (specifically with the rise of Deferred Rendering), traditional MSAA became technically difficult to implement efficiently. This led to the development of "Post-Processing" techniques.

- **Method:** FXAA is purely a screen-space effect. It analyses the final, fully rendered image as if it were a simple texture. It searches for high-contrast edges (which likely represent aliasing) and intelligently blurs the pixels along that edge direction to smooth them out.
- **Pros:** It is extremely fast and computationally cheap. Crucially, it works on everything in the imagegeometry, textures, shadows, and specular highlightsregardless of scene complexity.
- **Cons:** Because it is essentially a smart blur filter, it can sometimes blur fine texture details, making the overall game image look slightly "soft." It is not mathematically "correct" like MSAA, but rather a visual trick.

**Implementation in Three.js (EffectComposer):** If you utilise Post-Processing pipelines (like Bloom or Depth of Field), you often lose access to the built-in hardware MSAA. In these cases, you must add FXAA manually as the final pass in your composer chain.

```
import { EffectComposer } from 'three/examples/jsm/postprocessing/EffectComposer.js';
import { ShaderPass } from 'three/examples/jsm/postprocessing/ShaderPass.js';
import { FXAAShader } from 'three/examples/jsm/shaders/FXAAShader.js';

// 1. Setup Composer
const composer = new EffectComposer( renderer );

// 2. Add FXAA Pass
const fxaaPass = new ShaderPass( FXAAShader );

// CRITICAL: You must tell the shader the resolution of your screen
// so it knows how big 'one pixel' is to calculate the blur correctly.
const pixelRatio = renderer.getPixelRatio();
fxaaPass.material.uniforms[ 'resolution' ].value.x = 1 / ( window.innerWidth *
↪  pixelRatio );
fxaaPass.material.uniforms[ 'resolution' ].value.y = 1 / ( window.innerHeight *
↪  pixelRatio );

composer.addPass( fxaaPass );
```

Table 1: Comparison of Anti-Aliasing Techniques

| Technique | Mechanism | Pros | Cons |
|---|---|---|---|
| **SSAA** | Render at high res, shrink down. | Perfect quality. | Ultra slow (4× cost). |
| **MSAA** | Multiple depth samples, single colour sample. | Good compromise. Sharp textures. | Heavy memory usage. Hard with deferred shading. |
| **FXAA** | Intelligent blur filter on final image. | Very fast. Smoothes everything. | Can blur texture details. |

# 4 Efficient Mesh Construction: VBOs and IBOs

While high-level primitives like `THREE.BoxGeometry` are convenient, real-world graphics programming often requires constructing custom meshes from scratchwhether for procedural terrain, custom data visualisations, or optimised game assets. To do this efficiently, we must understand the underlying hardware structures: the **Vertex Buffer Object (VBO)** and the **Index Buffer Object (IBO)**.

## 4.1 The Problem: Data Redundancy

Consider a simple square made of two triangles. Naively, we might define this by listing the three vertices for the first triangle, followed by the three vertices for the second triangle.

- Triangle A: vertices $(0, 1, 2)$
- Triangle B: vertices $(2, 3, 0)$

Notice that vertices 0 and 2 are shared between the two triangles. In a naive approach (often called "soup" geometry), we would duplicate the full data for these verticesposition, normal, UVs, and colorin memory.

For a single square, this redundancy is negligible. However, for a complex character mesh with 50,000 connected triangles, each vertex is typically shared by an average of 6 triangles. Duplicating that data leads to massive bloat in Video RAM (VRAM) and forces the GPU to process the same vertex multiple times, wasting valuable computational power.

## 4.2 The Solution: The Index Buffer (IBO)

We solve this by decoupling the *geometry data* from the *connectivity logic*.

1. **The Data Pool (VBO):** We store each unique vertex exactly once in a linear array called a **Vertex Buffer Object**. This is our "palette" of available points.
2. **The Recipe (IBO):** We create a separate list of simple integersthe **Index Buffer Object**that acts as a set of instructions. It tells the GPU: "To build the first triangle, look up index 0, then 1, then 2. For the next triangle, reuse index 2, look up new index 3, and reuse index 0."

**Mental Model: The "Paint-by-Numbers" Analogy**

Think of the **VBO** as a palette of paint colors (the raw data).
Think of the **IBO** as the paint-by-numbers canvas (the instructions).
Instead of buying a new tube of "Blue" every time you need to paint a blue spot, you buy one tube and reference it repeatedly. The IBO allows the GPU to "reference" the heavy vertex data multiple times without copying it.

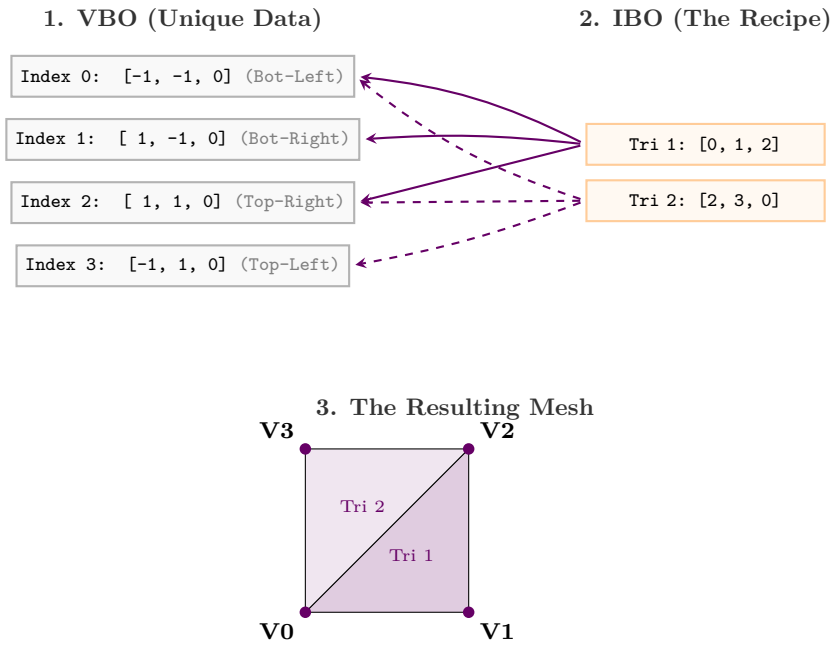



Figure 10: The relationship between VBO and IBO. The IBO "points" to data in the VBO, allowing vertices 0 and 2 to be reused for both triangles.

## 4.3 Implementation: Creating a Custom Mesh

In `three.js`, we interact with these buffers using the `BufferGeometry` class. We define data in Typed Arrays (which map directly to C++ memory structures) and attach them as attributes.

```
// 1. Define Unique Vertices (VBO Data)
// We only need 4 vertices to make a square (instead of 6)
const vertices = new Float32Array([
    -1.0, -1.0,  0.0, // V0: Bottom-Left
     1.0, -1.0,  0.0, // V1: Bottom-Right
     1.0,  1.0,  0.0, // V2: Top-Right
    -1.0,  1.0,  0.0  // V3: Top-Left
]);

// 2. Define Connectivity (IBO Data)
// We define two triangles referencing the vertex indices above
const indices = [
    0, 1, 2,  // Triangle 1 uses V0, V1, V2
    2, 3, 0   // Triangle 2 uses V2, V3, V0 (Reusing 0 and 2)
];

// 3. Create the BufferGeometry container
const geometry = new THREE.BufferGeometry();

// 4. Attach the VBO
// '3' indicates that each vertex has 3 components (x, y, z)
geometry.setAttribute('position', new THREE.BufferAttribute(vertices, 3));

// 5. Attach the IBO
geometry.setIndex(indices);

// 6. Create the Mesh
const mesh = new THREE.Mesh(geometry, new THREE.MeshBasicMaterial({color:
↪  0xff0000}));
scene.add(mesh);
```

## 5 The Architecture of Meshing & Texturing

Understanding how meshes and textures interact requires looking at the rendering pipeline as a whole. It is a collaborative effort between the **CPU** (Central Processing Unit), which acts as the "Architect" managing assets and logic , and the **GPU** (Graphics Processing Unit), which acts as the "Worker" performing massive parallel computation.

## 5.1 The Pipeline: From Logic to Pixels

The rendering process is not instantaneous; it is a pipeline where data transforms from abstract code into colored pixels. The following diagram illustrates the complete journey of mesh and texture data:

1. **Setup (CPU - The Architect):** Before any drawing occurs, the CPU must prepare the blueprints. It loads texture images into RAM and organizes geometric data (positions and UVs) into **Vertex Buffer Objects (VBOs)**. This is akin to an architect gathering materials before construction begins.

2. **Transfer (The Bridge):** When `renderer.render()` is called, the CPU issues a draw call. This effectively "wakes up" the GPU. Geometric attributes are streamed to the Vertex Shader , and texture units are bound to specific sampler uniforms, linking the image data to the shader logic.

3. **Rasterisation (GPU - The Interpolator):** Once the Vertex Shader has positioned the triangle corners, the GPU's fixed-function hardware takes over. It "rasterises" the shape, breaking it down into individual fragments (potential pixels). Crucially, it mathematically **interpolates** the UV coordinates from the three corners smoothly across the entire face of the triangle, ensuring every pixel has a unique, precise coordinate.

4. **Sampling (GPU - The Fragment Shader):** Finally, the Fragment Shader executes for every pixel. It receives the interpolated UV coordinate and uses it to "lookup" (sample) the corresponding color value from the texture stored in VRAM. This is the "magic trick" that paints 2D detail onto 3D geometry.

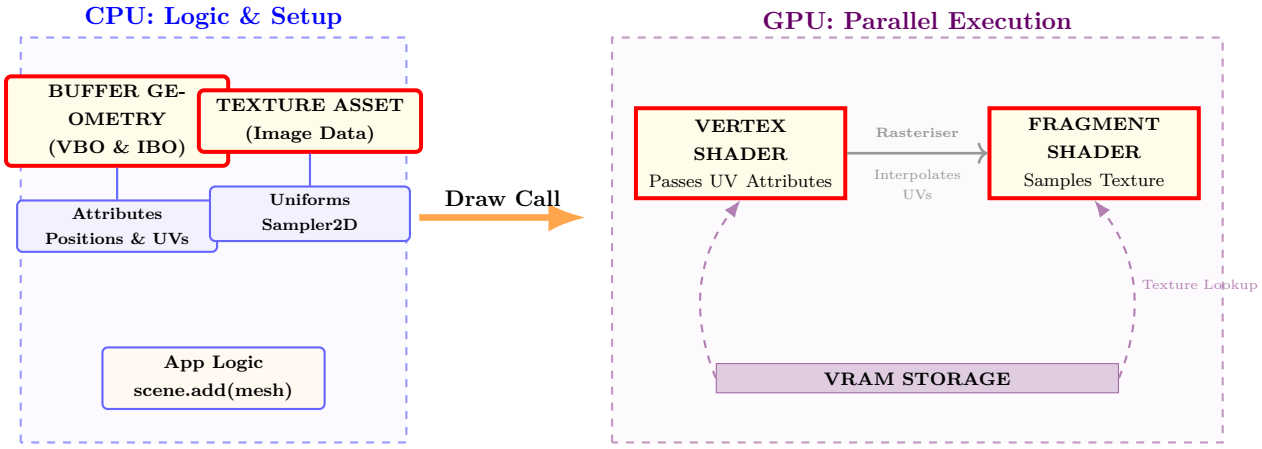



Figure 11: The Architectural Pipeline. Note how the Rasteriser sits between the shaders, interpolating data (like UVs) before the Fragment Shader runs.

### 5.2 Texture Modulation: The Mathematics of Surface Detail

Often, beginners assume a texture simply "pastes" an image onto a shape. In reality, the Fragment Shader performs a mathematical operation called **Modulation**.

The shader treats the texture color as the object's base "paint" (Albedo). It then multiplies this color by the calculated light intensity.

$$C_{final} = C_{texture} \times (I_{ambient} + I_{diffuse} + I_{specular})$$

This ensures that even a flat photo of a brick wall responds realistically to shadows and highlights in the 3D scene. If the light intensity is 0 (shadow), the result is black, regardless of the texture color.

## 6 Advanced Texture Applications: Beyond Simple Colour

While we have primarily discussed textures as a way to apply colour (Albedo) to a surface, their utility extends far beyond simple painting. In modern computer graphics, textures are treated as generic 2D data containers. We can store normals, heights, roughness values, or even entire lighting environments within them. This section explores three critical applications where textures drive geometry and lighting effects.

### 6.1 Billboarding: Cheating 3D Complexity

**The Concept:** Rendering fully 3D geometry for every object in a scene is computationally expensive. Imagine a forest with 10,000 trees or a particle system with 50,000 smoke puffs. If each tree were a detailed 3D mesh, the vertex count would be unmanageable.

**Billboarding** is an optimisation technique where we represent a complex 3D object using a single flat quadrilateral (a "quad" made of 2 triangles) with a transparent texture applied to it. To maintain the illusion of volume, this quad is programmed to automatically rotate so that it always faces the camera.

**The Mathematics:** To implement a billboard, we must calculate a specific model matrix for the quad.

1. Calculate the **Look Vector** ($\vec{D}$), which is the normalised vector pointing from the object's position to the camera's position:

$$\vec{D} = \text{normalize}(P_{camera} - P_{object})$$

2. Construct a rotation matrix that aligns the object's surface normal ($\vec{N}$) with this Look Vector ($\vec{D}$). In many engines, this is simplified by stripping the rotation component from the camera's View Matrix and applying its inverse to the object.

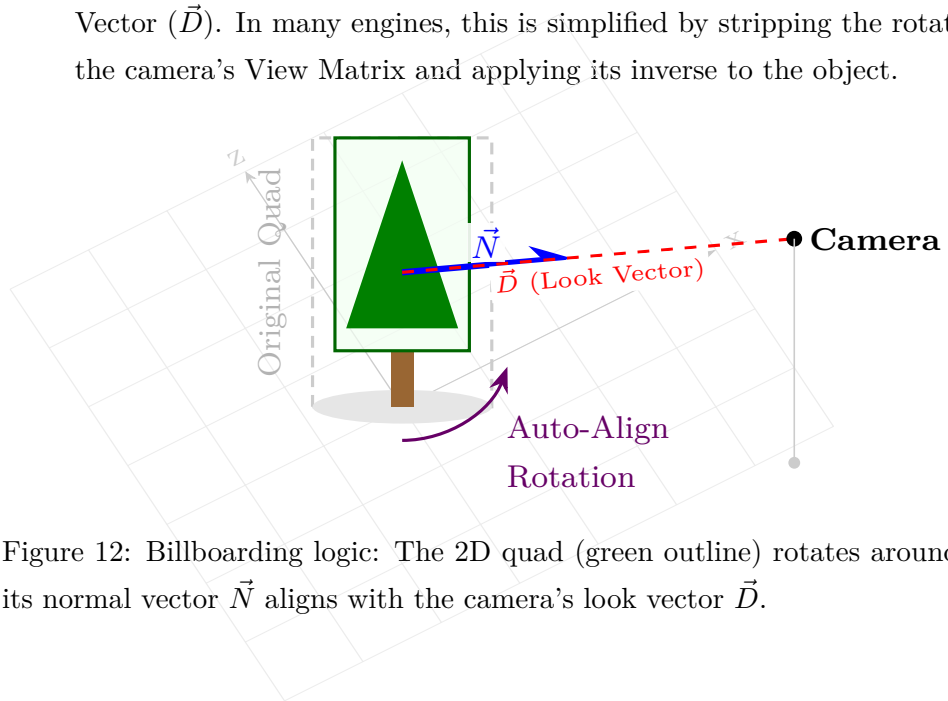



Figure 12: Billboarding logic: The 2D quad (green outline) rotates around the vertical axis so its normal vector $\vec{N}$ aligns with the camera's look vector $\vec{D}$.

## 6.2 Normal Mapping: The Illusion of Detail

One of the most significant advancements in real-time rendering is **Normal Mapping**. It addresses the conflict between performance (low polygon count) and visual fidelity (high detail).

**The Problem:** Geometric detail requires vertices. Modelling every crack in a brick wall or every pore on a character's face would require millions of triangles, which is impossible to render at 60 FPS.

**The Solution:** We cheat. Lighting calculations (like Phong shading) depend heavily on the **Normal Vector** ($\vec{N}$) of the surface. Usually, this normal is derived from the geometry itself (perpendicular to the triangle face). With Normal Mapping, we tell the Fragment Shader to ignore the real geometric normal and instead look up a "fake" normal from a special texture called a **Normal Map**.

**How it Works:**

- A Normal Map stores $(x, y, z)$ vector data in the $(r, g, b)$ colour channels of an image.
- Since normal vectors have components ranging from $[-1, 1]$ and colours range from $[0, 1]$, we remap the data:
$$\vec{N}_{xyz} = (\text{Color}_{rgb} \times 2.0) - 1.0$$
- A flat surface (Normal $0, 0, 1$) corresponds to the colour RGB $(0.5, 0.5, 1.0)$, which creates the characteristic "periwinkle blue" colour of normal maps.

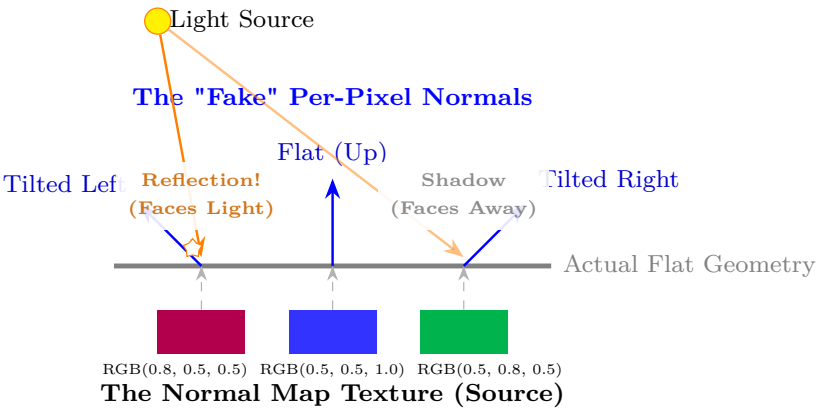



Figure 13: Normal Mapping tricks the lighting engine. The geometry is flat (saving polygons), but the lighting behaves as if the surface is bumpy because we manipulate the normal vector per-pixel.

## 6.3 Environment Mapping: Real-Time Reflections

Reflective surfaces like chrome, water, or mirrors are difficult to render because they need to show the surrounding world. Ray-tracing the entire scene for every pixel is too slow for real-time graphics. Instead, we use **Environment Mapping** (or Reflection Mapping).

**The Concept:** We treat the background environment as a texture wrapped around the entire scene. Usually, this is stored as a **Cube Map**, which is a special texture composed of 6 square images (Up, Down, Left, Right, Front, Back) that form a seamless box around the camera.

**The Implementation:** To render a reflective object:

1. We calculate the vector from the camera (Eye) to the surface point ($I$).
2. We calculate the reflection vector ($R$) by bouncing $I$ off the surface normal $N$:

$$R = \text{reflect}(I, N)$$

3. We use this 3D vector $R$ as a lookup coordinate to sample the Cube Map. The GPU calculates which face of the cube the vector hits and returns that pixel's colour.

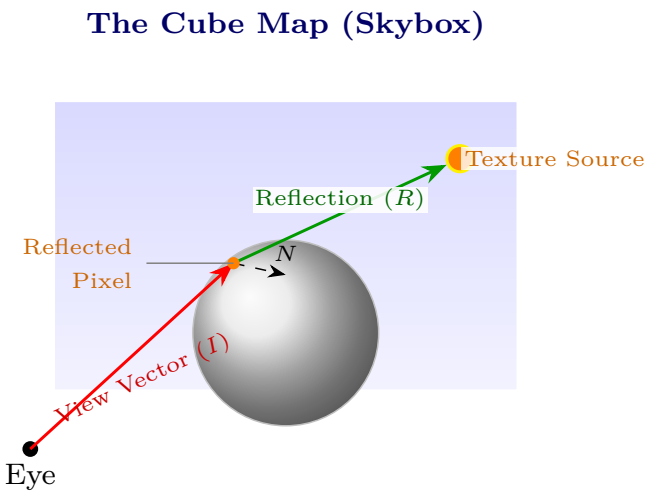



Figure 14: Environment Mapping using a Cube Map. The Reflection Vector ($R$) is used to fetch a colour from the "sky" texture, which is then painted onto the object surface.

## 7 Reflective Questions: Beyond the Surface

As we conclude our exploration of geometry and texturing, consider the following questions. They are designed to help you look past the immediate mechanics of WebGL and think about the deeper challenges involved in rendering complex virtual worlds.

- **The Data Bottleneck (Memory vs. Detail):** We learned that models are made of vertices stored in memory. A high-quality character might have 50,000 triangles. *If an open-world game needs to render a forest with 10,000 trees, storing a unique copy of every single leaf and branch would crash the GPU's memory instantly. How can we render the same tree 10,000 times with different positions and rotations without duplicating the raw vertex data? (Search term: Instanced Rendering).*

- **The Illusion of Depth (Normal Mapping):** We use textures to paint colour onto a flat surface, but the surface geometry remains flat. If you look at a brick wall texture from the side, it looks like a smooth sheet of paper. *Creating millions of tiny triangles for every crack in a brick is too slow. Is there a way to use a special kind of texture to "trick" the lighting calculations into thinking the surface is bumpy, without actually adding more vertices? (Search term: Normal Mapping).*

- **The Infinite Canvas (Procedural Generation):** We manually defined UV coordinates to map an image to a shape. *What happens if you want to model a planet where the terrain stretches on forever? You can't paint a texture big enough for that. How could we use mathematics (like sine waves or noise functions) inside the shader to generate terrain features*

*and colours on the fly, without using any texture images at all?.*

- **The Distance Problem (Level of Detail):** We discussed using Mipmaps to lower texture resolution for distant objects. *Shouldn't we do the same for geometry? It is a waste of processing power to calculate 50,000 triangles for a character that is only 10 pixels tall on the screen. How could a game engine dynamically swap out complex meshes for simpler ones as objects move further away? (Search term: Level of Detail / LOD).*

*"The details are not the details. They make the design."*
Charles Eames